



Amrita School of AI
COIMBATORE CAMPUS

UNIT 2 · SESSIONS 14-19

MPI

Nothing is shared. Everything is a message. That is both the cost and the safety.

Dr. Abhijith Anandakrishnan · Assistant Professor
23AID304 · Odd Semester 2026-27

Threads vs ranks

- **OpenMP**

- Threads share one address space
- Communication is a memory write
- Free, and dangerous
- **One node**

- **MPI**

- Processes, private memory
- Communication is explicit
- Verbose, and safe
- **The whole cluster**

Four lines of scaffolding

```
MPI_Init(&argc, &argv);  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);    // who am I  
MPI_Comm_size(MPI_COMM_WORLD, &size);    // how many of us  
...  
MPI_Finalize();
```

Same executable on every process. Each branches on its rank. **SPMD**.

Six arguments: what, who, which

```
MPI_Send(buf, count, type,  
         dest, tag, comm);
```

```
MPI_Recv(buf, count, type,  
         source, tag, comm,  
         &status);
```

A receive matches a send only when **source, tag and communicator** all agree.

`MPI_ANY_SOURCE` and `MPI_ANY_TAG` relax the first two.

Now the deadlock everyone writes.

This hangs

```
MPI_Send(sendbuf, n, MPI_DOUBLE, neighbour, 0, comm);  
MPI_Recv(recvbuf, n, MPI_DOUBLE, neighbour, 0, comm, &st);
```

Both sides call `Send` first. `Send` may block until a matching `Recv` is posted. Neither ever reaches its `Recv`.

It worked with small messages

Small messages get copied into an internal buffer, so `Send` returns immediately.

Cross the **eager limit** (tens of KB) and the implementation switches to rendezvous.

It then deadlocks on a bigger problem, in production, having passed every test. ****Never rely on buffering.****

Order it, combine it, or overlap it

- **Order by rank**

```
if (rank % 2) { Send;
Recv; }
else          { Recv;
Send; }
```

- **Sendrecv**

```
MPI_Sendrecv(...);
```

One call, no deadlock by construction.

- **Non-blocking**

```
MPI_Irecv(...);
MPI_Isend(...);
compute_interior();
MPI_Waitall(...);
```

Also
hides the network behind real work

THE CATCH

Non-blocking means the buffer is not yours

After `MPI_Isend` returns, the message has **not** been sent.

Touching `sendbuf` before `MPI_Wait` completes corrupts the message. Intermittently. On someone else's machine.

COLLECTIVES

Called by everyone, and faster than your version

CALL	DOES
Bcast	one → all
Scatter	split across ranks
Gather	collect at root
Reduce	combine at root
Allreduce	combine → all
Alltoall	everyone → everyone

Allreduce on P ranks: $\log_2 P$ steps.

Hand-rolled gather at rank 0: P steps, and it serialises on one network link.

$$T(n) = \alpha + n / \beta$$

Latency is paid **per message**. One 1 MB message beats a thousand 1 KB messages.

Below ~20 KB you are paying for latency, not bytes

$$\alpha = 2 \mu\text{s}, \beta = 10 \text{ GB/s}$$

$$n = \alpha \times \beta = \mathbf{20\ 000 \text{ bytes}}$$

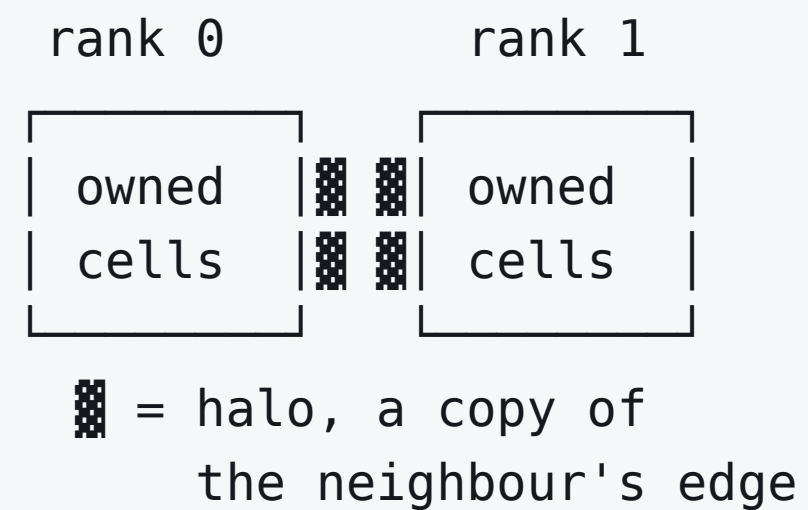
Below it, doubling the message size costs almost nothing.

This is why **aggregating small messages** is the first thing to try.

THE PATTERN BEHIND MOST SCIENTIFIC COMPUTING

Domain decomposition

Own the interior, borrow the edge



Refresh the halo **once per step**, not once per cell.

`MPI_PROC_NULL` removes every boundary special case:
sends to it silently succeed.

Why 2-D decomposition beats 1-D

- **1-D strips**

Each rank: N/P rows

Communicates $2N$ values

ratio = $2P / N$

- **2-D blocks**

Each rank: $(N/\sqrt{P})^2$ block

Communicates $4N/\sqrt{P}$ values

ratio = $4\sqrt{P} / N$

**N = 1024, P = 64: four times less
communication.**

Same computation. The advantage grows as $\sqrt{P}$, so at scale it is a requirement, not an optimisation.

Why large solvers are iterative

Direct (LU, Cholesky)

Fill-in: zeros become non-zeros. Memory grows, the pattern turns irregular, balance is hard.

Iterative (CG, Jacobi)

Each step = halo exchange + sparse matvec + two `Allreduce`.

Parallelises predictably.

Hybrid: MPI between nodes, OpenMP within

WHAT REAL CODES DO

One rank per node, 96 threads inside.

```
export OMP_NUM_THREADS=96
export OMP_PROC_BIND=close
srun --nodes=3 --ntasks-per-node=1 \
    --cpus-per-task=96 ./hybrid
```

Fewer ranks → fewer, larger messages → less α .

Halos and read-only data shared once per node, not duplicated per rank.

‘OMP_PROC_BIND’ is not optional. Unpinned timings are not reproducible.

Next: the accelerator

Same ideas, 220 000 threads.